LINUX III · REDES Y SERVICIOS

Administración de Servicios en Linux

Arquitectura · systemd · POSIX · Seguridad · Hardening

systemd · POSIX · cgroups · journalctl · hardening

TEORÍA | PRÁCTICA | SEGURIDAD | 3 HORAS

Unidades 1–2: Gestión avanzada de servicios con systemd

Desmitificando los Conceptos Clave

Definición de Ingeniería: Un servicio es un programa ejecutado en background, que provee funcionalidad persistente al sistema o red, y es administrado por el gestor de inicialización del OS.

CONCEPTO	DEFINICIÓN CENTRAL	¿QUIÉN LO GESTIONA?	EJEMPLO
PROCESO	Programa en ejecución con PID y espacio de memoria propio.	El Kernel Linux	`ls, bash, ps`
DAEMON	Proceso en background, desvinculado de la terminal, esperando peticiones.	El propio binario / Kernel	`sshd, nginx`
SERVICIO	Un daemon envuelto en políticas de ciclo de vida, dependencias y logs.	Gestor de inicio (systemd)	`ssh.service`

💡 Un daemon puede existir sin ser un servicio (ejecutado manualmente con &), pero systemd le otorga persistencia, reinicio y control de dependencias.

Modelo de Capas del Sistema Linux

5	SERVICIOS	nginx, sshd, docker — gobernados por systemd
4	DAEMONS	Procesos en background desvinculados de terminal
3	PROCESOS	Unidad básica de ejecución — tienen PID y contexto
2	KERNEL	Gestiona CPU, memoria, red y dispositivos físicos
1	HARDWARE	CPU · RAM · Disco · Red — recursos físicos del nodo

El Estándar POSIX — Por qué Importa

POSIX (Portable Operating System Interface — IEEE 1003) es el contrato que garantiza que el software interactúa de forma predecible con cualquier sistema UNIX-like.

Gestión de Procesos

- `fork()` — duplica un proceso
- `exec()` — reemplaza imagen del proceso
- `wait()` — espera la muerte de un hijo
- `kill()` — envía señales entre procesos

I/O y Sistema de Archivos

- `open()` — abre un descriptor de archivo
- `read()` / `write()` — lectura y escritura
- `close()` — libera el descriptor
- Modelo: todo es un archivo

Permisos y Señales

- `chmod` / `chown` — control de acceso
- `SIGTERM` — terminación suave
- `SIGKILL` — terminación forzada
- `SIGHUP` — recarga de configuración

Nginx (Daemon) → glibc (Librería C) → Syscalls POSIX → Kernel Linux → Hardware

Secuencia de Arranque del Sistema

01

BIOS / UEFI

Inicialización de hardware. POST y detección de dispositivos de arranque.

02

Bootloader (GRUB)

Carga el Kernel Linux en RAM. Pasa parámetros de arranque al kernel.

03

Kernel Linux

Inicializa estructuras internas, monta raíz, carga drivers esenciales.

04

systemd (PID 1)

El Kernel cede control al espacio de usuario. systemd orquesta el resto.

05

Servicios

systemd lee targets y levanta: red, SSH, web, Docker y demás servicios.

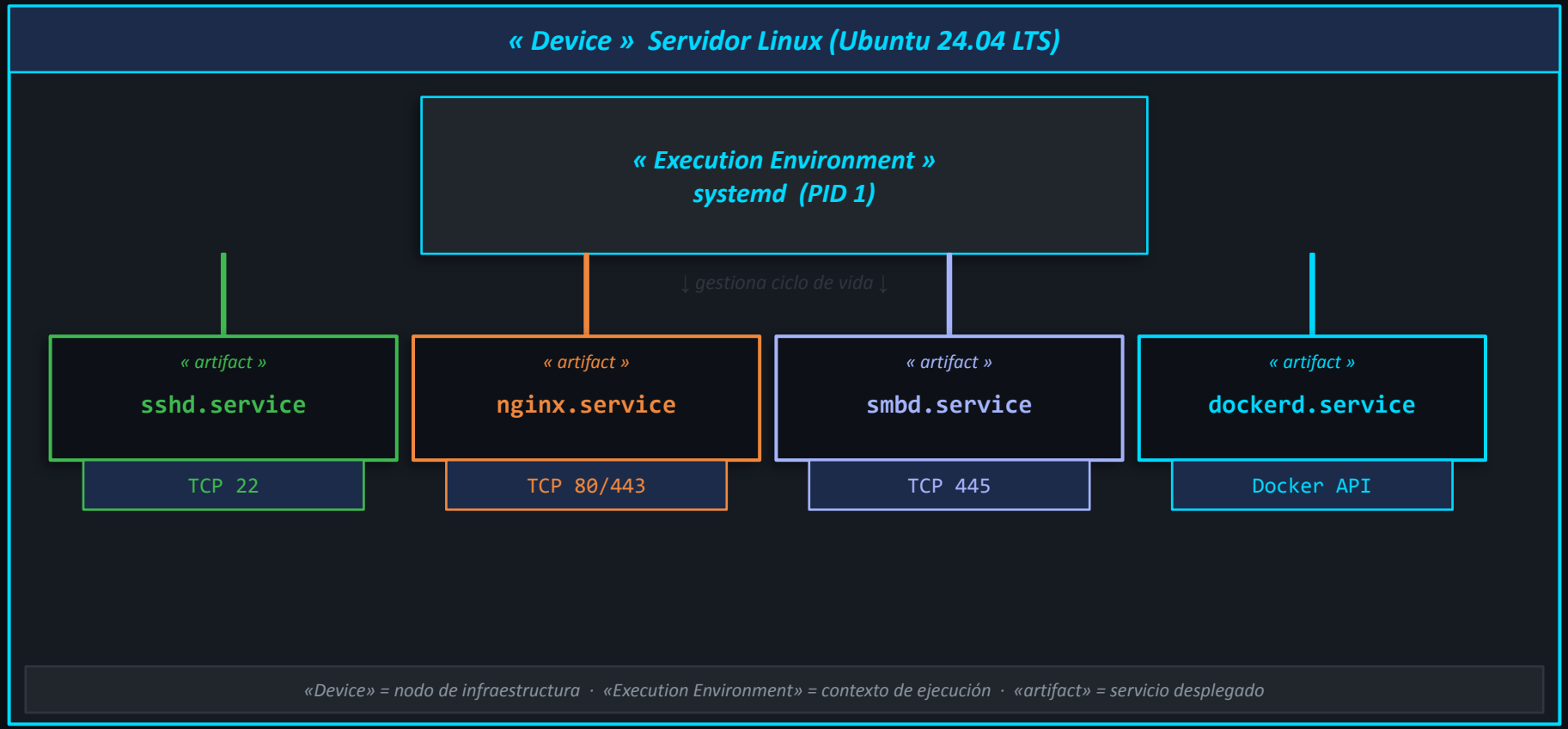
Demostración en vivo:

```
$ ps -p 1
   PID TTY      CMD
     1 ?        /lib/systemd/systemd
```

¿Por qué PID 1 es especial?

- Ancestro de todos los procesos de usuario
- Adopta procesos huérfanos (reparenting)
- Si muere, el kernel emite un kernel panic
- Gestiona el apagado y reinicio del sistema

Diagrama UML de Despliegue



Anatomía de un Unit File .service

/etc/systemd/system/mi-app.service

```
[Unit]
Description=Mi Aplicación Web
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/app/main.py
Restart=always
User=www-data
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

[Unit]

- Description — nombre legible del servicio
- After — orden de inicio (network.target)
- Requires — dependencias fuertes (opcional)

💡 *Modelo declarativo: definimos QUÉ, no CÓMO*

[Service]

- ExecStart — comando de arranque
- Restart — política: always / on-failure
- User — usuario de ejecución (principio mínimo privilegio)

[Install]

- WantedBy — target de activación
- multi-user.target = modo servidor (sin GUI)

💡 *Activo con: systemctl enable mi-app*

Arquitectura de systemd — Tipos de Units

PID 1

El primer proceso del sistema operativo

```
$ ps -p 1
1  systemd
```

Responsabilidades:

- Inicia todos los servicios del sistema
- Monta filesystems en el arranque
- Gestiona dependencias entre servicios
- Controla apagado y reinicio del sistema

Tipos de Units

.service
Daemons y servicios del sistema

.socket
Activación por socket de red

.timer
Tareas programadas (reemplaza cron)

.mount
Montaje de filesystems

.target
Agrupación de units (runlevels)

.path
Monitoreo y reacción a rutas

Hardening de Servicios con systemd

systemd permite aplicar sandboxing y aislamiento a nivel de servicio — sin código extra ni contenedores adicionales.

PARÁMETRO systemd	¿QUÉ HACE?	BENEFICIO DE SEGURIDAD
`NoNewPrivileges=true`	Impide que el proceso y sus hijos obtengan nuevos privilegios (ignora suid).	Evita escalada de privilegios post-compromiso.
`PrivateTmp=true`	Crea un /tmp y /var/tmp aislado exclusivo para este servicio.	Evita ataques de colisión y robo de temporales entre servicios.
`ProtectSystem=strict`	Monta el sistema de archivos (excepto datos) como solo lectura.	El servicio no puede alterar binarios ni configuraciones del OS.
`MemoryMax=200M`	Usa cgroups v2 para limitar el consumo máximo de RAM del proceso.	Previene caídas del sistema por Out Of Memory (OOM).
`ProtectHome=true`	Hace los directorios /home, /root y /run/user invisibles para el servicio.	El servicio no puede acceder a datos personales de usuarios.

systemctl — Control de Servicios

```
$ systemctl list-units --type=service --state=running
```

Auditoría de servicios activos

```
$ systemctl status ssh
```

Estado + Main PID + árbol de cgroups

```
$ systemctl cat ssh
```

Inspección del unit file del servicio

```
$ sudo systemctl start nginx
```

Iniciar un servicio manualmente

```
$ sudo systemctl restart nginx
```

Reiniciar completamente el servicio

```
$ sudo systemctl enable nginx
```

Activar arranque automático al boot

```
$ sudo systemctl daemon-reload
```

Recargar unit files tras modificarlos

```
$ systemd-analyze blame
```

Profiling del tiempo de arranque

Observabilidad Centralizada: journalctl

Antes de systemd

- /var/log/syslog
- /var/log/messages
- /var/log/auth.log
- /var/log/daemon.log
- /var/log/kern.log

Logs dispersos — difícil diagnóstico

→

Con journald (systemd)

```
$ journalctl -u ssh -n 20 --no-pager
```

Últimas 20 líneas del servicio

```
$ journalctl -u nginx -f
```

Seguimiento en tiempo real

```
$ journalctl -p err --since today
```

Errores del día de hoy

```
$ journalctl --since '1 hour ago'
```

Logs de la última hora

```
$ journalctl -u ssh --output=json
```

Formato JSON para integración

Laboratorio 1: Exploración del Ecosistema systemd

Objetivo: Transicionar de la teoría del PID 1 a la práctica en terminal real · Duración: 40–50 min

01 **Auditoría de servicios activos**

```
$ systemctl list-units --type=service --state=running
```

❓ ¿Cuántos son del sistema vs. aplicaciones instaladas?

02 **Análisis del ciclo de vida**

```
$ systemctl status ssh
```

❓ Identificar: Main PID, estado y árbol de cgroups

03 **Ingeniería inversa del servicio**

```
$ systemctl cat ssh
```

❓ ¿Qué dependencias tiene SSH? ¿Qué usuario usa?

04 **Auditoría de logs centralizados**

```
$ journalctl -u ssh -n 20 --no-pager
```

❓ ¿journald captura stdout y stderr del daemon?

05 **Profiling del arranque del sistema**

```
$ systemd-analyze blame
```

❓ ¿Qué servicio es el cuello de botella? ¿Por qué?

Todo se Administra con systemd

apache2.service

U2 — Servidor Web

sshd.service

U2 — Acceso Remoto

smbd.service

U2 — Samba / Archivos

systemd

PID 1 — Orquestador

docker.service

U3 — Contenedores

squid.service

U2 — Proxy

nftables.service

U2 — Firewall

REFLEXIÓN FINAL

Un servidor Linux moderno no es una colección caótica de programas.

Es un ecosistema estructurado de servicios gestionados por systemd. Cuando administramos Linux, orquestamos recursos, modelamos dependencias y aplicamos políticas de seguridad a través del PID 1.

Tarea para la Próxima Clase

```
$ systemctl list-units --type=service --state=running
```

1. ¿Cuántos servicios están corriendo activamente? ¿Cuáles son del sistema vs. aplicaciones?
2. ¿Cuál es el servicio con mayor tiempo de arranque? (systemd-analyze blame)
3. Ejecutar systemctl cat ssh — ¿qué parámetros de seguridad tiene configurados?
4. Agregar NoNewPrivileges=true a un servicio de prueba y verificar el efecto